

Tecnico Superiore per lo sviluppo di sistemi full stack per web, mobile e desktop
ITS Software Architect Specialist
NUOVA EDIZIONE

Biennio 2023-25

Dal banco alla scrivania: il mio percorso formativo e professionale

Davide Sala

Indice

Introduzione.....	3
Chi sono.....	3
Il mio ingresso nel mondo del lavoro.....	4
Esperienza in azienda.....	5
Progetti svolti.....	5
Android.....	5
Angular + Spring Boot	7
Competenze utilizzate.....	11
Hard skills	11
Soft skills	18
Sfide affrontate e lezioni imparate	19
Conclusioni e prospettive future	23
Sitografia	24

Introduzione

Chi sono

- **Chi sono**

Mi chiamo Sala Davide, ho 21 anni e ho conseguito il diploma di maturità presso l'Istituto di Istruzione Superiore Martino Bassi di Seregno (MB), indirizzo Sistemi Informativi Aziendali.

Dal terzo anno iniziai a capire che l'economia aziendale non faceva per me. Invece, l'informatica e la programmazione avevano attirato la mia attenzione e quindi una volta conseguito il diploma decisi di continuare i miei studi in quella direzione.

- **ITS Angelo Rizzoli**

Dopo il diploma ho scelto di iscrivermi al corso Software Architect presso l'ITS Angelo Rizzoli, un "istituto tecnologico superiore" nato nel 2010. Nonostante le mie conoscenze di programmazione fossero limitate il mio obiettivo era quello di esplorare questo nuovo mondo per capire se la mia curiosità si potesse trasformare in una vera passione professionale.

Il corso biennale si è rivelato un'esperienza formativa completa: la prima parte dedicata all'acquisizione delle competenze fondamentali, la seconda focalizzata sul consolidamento pratico delle basi apprese nella prima parte.

La formazione era strutturata in unità formative trasversali (UFT) e unità formative specialistiche (UFS), progettate per fornire sia competenze applicabili in ambito lavorativo che nella vita quotidiana. Il secondo anno del corso ha rappresentato una svolta significativa: dopo una fase iniziale di apprendimento teorico, le restanti 840 ore sono state dedicate allo stage aziendale, permettendomi di applicare le conoscenze acquisite in un contesto reale.

Il mio ingresso nel mondo del lavoro

Durante la ricerca dello stage alcuni colleghi di corso optavano anche per aziende che avrebbero richiesto competenze completamente diverse da quelle acquisite in ITS. Io invece ho scelto Object Method S.r.l., un'azienda che cercava proprio le competenze di Java e programmazione a oggetti che avevo studiato durante il primo anno del corso.

- **Azienda**



Object Method S.r.l., una software house con sede a Cernusco Sul Naviglio (MI) che vanta circa 30 dipendenti, molti dei quali lavorano in modalità smart-working.

L'azienda, attiva da oltre 20 anni nel settore della consulenza informatica, collabora stabilmente con scuole superiori e università per la formazione di nuovi talenti. I

clienti sono spesso PMI (piccole e medie imprese) ma a volte anche grandi realtà, come Esselunga e UniCredit, si affidano a Object Method per lo sviluppo di nuove funzionalità e per la manutenzione dei sistemi esistenti.

Nonostante la notevole distanza dalla mia residenza l'ufficio era facilmente raggiungibile con i mezzi pubblici. Dopo un colloquio con esito positivo, e un breve briefing su come sarebbe stato strutturato lo stage e la presentazione dei tutor e dei colleghi, iniziai il mio percorso di stage insieme a una manciata di altri studenti del mio corso.

Esperienza in azienda

Il 13 gennaio 2025 è ufficialmente iniziato il mio primo stage in azienda.

Le prime settimane sono state caratterizzate da esercitazioni con difficoltà crescente su algoritmi, strutture dati, query SQL con JOIN e test sulle nostre conoscenze di Spring Boot e Angular, permettendo ai tutor di valutare le nostre competenze di base.

Superata questa fase di assesment, ci sono stati assegnati progetti pratici più complessi, progettati per integrare le nostre competenze di sviluppo frontend e backend in soluzioni complete.

Progetti svolti

Alcuni dei progetti sviluppati durante il periodo di stage sono stati realizzati su iniziativa personale, mentre altri sono stati commissionati per dei clienti o per scopi individuali.

I progetti utilizzavano varie tecnologie in base alle esigenze e i principali sono quelli che seguono:

Android

- **Rally TSD** (per il Panda Raid 2025)

Descrizione: Questo progetto è stato commissionato a me e un altro ragazzo che come me stava svolgendo il suo percorso di stage con ITS. Lo scopo era quello di creare una app compatibile con i dispositivi mobili Android per uso personale del titolare, il quale essendo appassionato di rally occasionalmente prende parte ad alcune gare di **“Rally di precisione”** (Time Speed Distance).

In questo tipo di competizione non serve arrivare per primi al traguardo per vincere, invece serve seguire con precisione una serie di **note** (istruzioni che indicano la distanza da percorrere, la velocità media da mantenere e un'immagine che rappresenta la fine della nota) fornite dagli organizzatori dell'evento il giorno prima della gara. Per ogni secondo di anticipo o ritardo rispetto all'orario previsto di completamento di una nota vengono applicate penalità al punteggio.

In un evento del genere l'accuratezza del calcolo delle distanze e del tempo è essenziale e quindi

anche la app doveva rispecchiare queste esigenze.

Questo ha richiesto un'analisi accurata delle varie situazioni che si sarebbero potute verificare nel corso della gara, e trovare soluzioni adatte a gestirle.

Funzionamento: Sviluppata utilizzando Android Studio fa uso di alcune funzionalità integrate dei dispositivi su cui viene installata per calcolare il tempo e la distanza percorsa attraverso un cronometro e il servizio della posizione.

In base alla versione della SDK del dispositivo fa uso o della posizione ottenuta da un Location Provider. Per iniziare a utilizzare la app serve inserire una lista di "note" dall'interfaccia della home, dopodiché viene calcolato il tempo necessario per percorrere ogni nota e premere il bottone con il simbolo "play". All'avvio del cronometro viene registrato il timestamp di partenza e vengono calcolati i timestamp di arrivo previsti per ogni nota.

Utilizzando le informazioni fornite dal provider della posizione durante il tragitto, calcola la distanza mancante e la velocità attuale per stimare un timestamp di arrivo effettivo. Se c'è discrepanza tra il timestamp di arrivo previsto e quello stimato il programma calcolerà la velocità da raggiungere necessaria per completare la nota nell'orario stabilito.

Al raggiungimento del termine di una nota è necessario solamente cliccare una qualsiasi parte dello schermo del dispositivo per passare alla nota successiva. La scelta di usare l'intero schermo come un bottone è nata dal fatto che le gare si svolgono spesso su tratti di strada accidentati e quindi cliccare in modo preciso sarebbe complicato per via delle vibrazioni.

Al click viene confrontato il timestamp di arrivo previsto e quello effettivo e vengono mostrati eventuali ritardi o anticipi. Se nelle impostazioni è stata inserita la quantità applicata come penalità per ogni secondo di differenza viene mostrata al pilota anche una stima del suo punteggio attuale.

Una delle ultime aggiunte al progetto riguardava appunto le impostazioni.

Cliccando l'icona di un ingranaggio nella schermata home si possono cambiare le costanti che il programma utilizza, tra cui la precisione minima richiesta dai location provider e il tempo massimo che deve passare tra gli aggiornamenti della posizione.

Sempre nella sezione delle impostazioni, gli elementi dell'interfaccia possono essere nascosti a

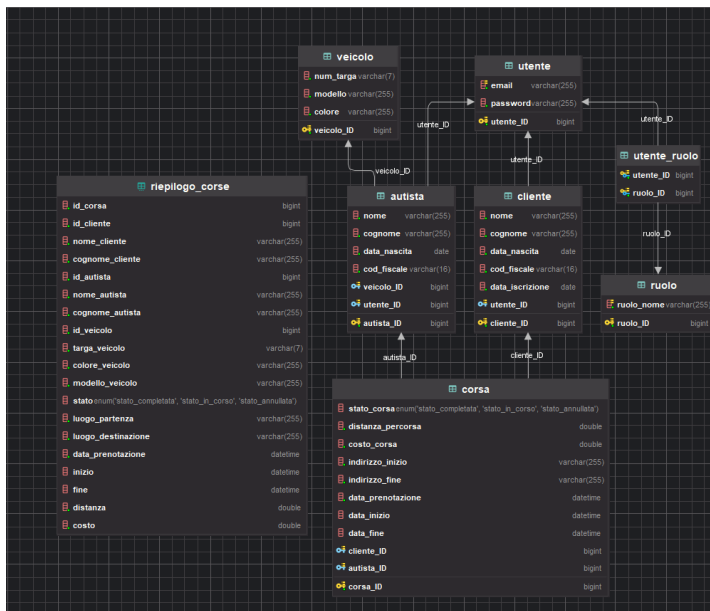
discrezione del pilota, per evitare di avere lo schermo pieno delle informazioni che non sono di suo interesse.

Angular + Spring Boot

- **Esercitazione per formazione** (su database GestioneTaxi)

Descrizione: Un'esercitazione svolta nella fase iniziale dello stage aveva richiesto la creazione di un database per un ipotetico servizio di Taxi. Questo database è stato ampliato per lo sviluppo di questo progetto, affiancandolo a un backend Spring Boot e un interfaccia web Angular.

Il database utilizzato conteneva i dati provenienti da autisti, clienti, veicoli, corse e utenti.



Il progetto è strutturato utilizzando il pattern **MVC** per essere più ordinato e leggibile, oltre che più mantenibile, e fa uso di **Mapstruct** e **Lombok** per ridurre il boilercode.

Per la sicurezza usai solo i JWT, i quali vengono intercettati da una classe **AccessFilter** per verificare le informazioni al suo interno.

Il database fa utilizzo di semplici id numerici invece che gli uuid utilizzati nei progetti successivi.

Funzionamento: Il frontend è formato da diverse pagine, ogni pagina mostra il contenuto di una tabella diversa, in aggiunta è presente una pagina “summary” che contiene i valori presi da tutte le tabelle attraverso una View. Per visualizzare i dati non è obbligatorio essere un utente loggato ma tutte e chiamate REST che potrebbero effettuare modifiche richiedono un token con un livello di autorizzazione adeguato.

Gli errori a runtime e i warning vengono catturati automaticamente da una classe intercettore e dopo essere loggati innescano degli alert che mostrano all'utente cosa non va.

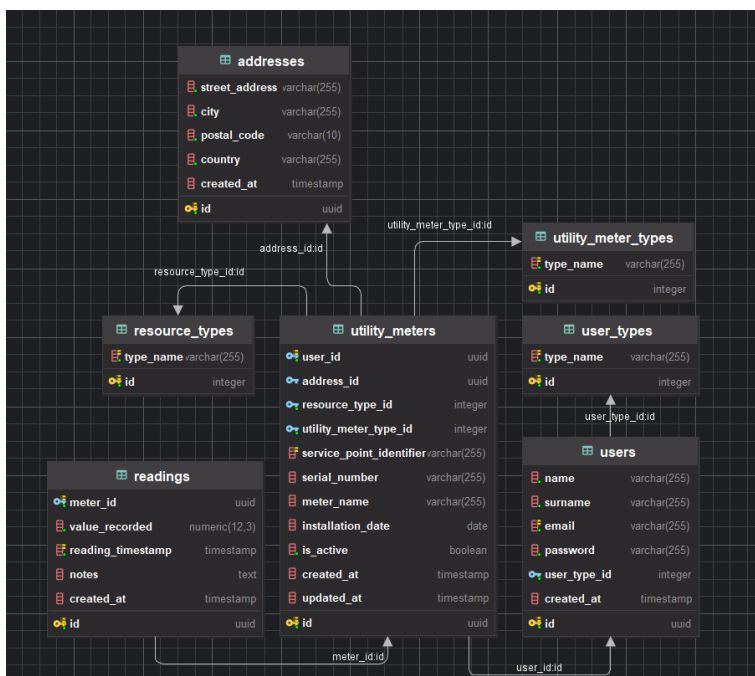
Essendo il mio primo progetto mi ha permesso di sperimentare con l'implementazione di nuove

funzionalità seguendo le linee guida dei tutor, tra cui il **code-coverage** attraverso l'uso di unit e integration tests.

- **Gestione utenze gas, energia e acqua** (è ancora un WIP)

Descrizione: Il secondo progetto svolto in stage è ancora in corso d'opera. Fa uso di una parte frontend Angular (con typescript) che comunica con un backend Spring Boot Java e un database Postgres hostato con Docker.

L'obiettivo era quello di gestire i dati provenienti dai contatori delle utenze e immagazzinare le letture periodiche per permettere ai proprietari di visualizzarle sottoforma di grafici a linee per farsi un'idea migliore dei propri consumi nel tempo. Se i contatori sono manuali le letture vengono inserite dall'utente, se automatici ogni giorno manderanno i dati delle letture in modo autonomo.



Funzionamento: Il database è composto da più tabelle collegate a una centrale.

Attualmente il programma permette di effettuare il login o la registrazione degli utenti e di visionare i propri dati che sono già inseriti nel database.

La sicurezza degli accessi è attualmente gestita tramite

l'utilizzo di JWT. Per impedire l'accesso ai dati altrui il sistema estrae in automatico l'uuid dell'utente dai claims del token e lo usa nelle query per restituire solo i dati relativi al proprietario del token.

Allo stesso modo le modifiche possono essere effettuate solo sui propri dati, a meno che il ruolo indicato nel payload del token non sia "admin", in tal caso le restrizioni non vengono applicate per dare più libertà.

Le password sono state protette utilizzando l'algoritmo di hashing **BCrypt**.

Questo algoritmo non cifra le password, le trasforma in modo irreversibile (hashing). Durante questo processo viene utilizzando un codice randomico, chiamato “**salt**”, grazie al quale due hash non saranno mai uguali anche se originati dalla stessa parola. L’unico modo per ottenere lo stesso risultato è usare lo stesso salt originale. Questo rende sicure le password con hash perché anche se venissero lette da qualcuno non sarebbero in grado di estrarre la password effettiva.

Alcune funzioni non sono ancora totalmente implementate ma ho in programma di continuare a lavorare su questo progetto in futuro. Una delle aggiunte che ho pianificato riguarda le chiamate ai singoli lettori delle utenze. Tutti i lettori che risultano come “automatici”, riceveranno una chiamata di tipo GET periodica utilizzando Quartz.

Quartz è uno scheduler utile per ripetere l’esecuzione di parti di codice in modo regolare. Questo permetterà di ottenere i dati dai lettori in modo automatico.

Per quel che riguarda il frontend, la grafica con i fogli di stile CSS non è ancora presente, ma ho in programma di utilizzare TailwindCSS dato che grazie al progetto CCE2025 sono riuscito a imparare le basi su come farne uso.

- **CCE2025**

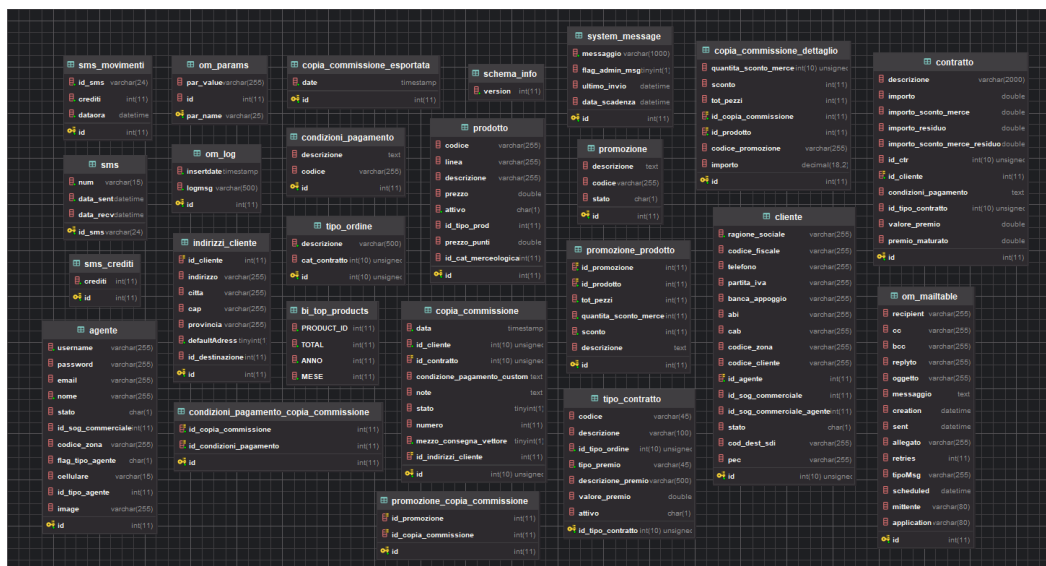
Descrizione: Un progetto che hanno affidato a noi stagisti in azienda, il nostro compito era quello di ammodernarlo poiché faceva utilizzo di versioni di java ormai in disuso.

Il programma serve la funzione di gestionale per le fatture di un’azienda che si occupa di vendita di accessori e prodotti per parrucchieri e saloni di bellezza.

Il mio ruolo nel team era quello di lavorare principalmente alla parte backend, in particolare sulla logica di creazione dei pdf delle copie-commissioni prese dal database e l’invio dei file via mail quando viene “chiusa” una bozza.

Funzionamento: Il database dal quale prendiamo i dati era privo di foreign key dichiarate e quindi le tabelle erano tutte scollegate tra loro. Inoltre, non ci era consentito modificare la struttura del

database poiché essendo collegato anche ad altri servizi a noi sconosciuti una qualsiasi modifica avrebbe potuto compromettere il loro funzionamento. Spettava a noi trovare i collegamenti e usarli per costruire le query utili a estrarre tutti i dati a noi necessari.



Le **copie commissioni** una volta create possono essere scaricate, create come bozza oppure essere “chiusure”.

Nel caso in cui si vuole chiudere una copia viene generato e salvato un pdf. In seguito, si innesca l’invio di una mail di notifica sia all’agente che ha chiuso la copia sia all’amministrazione. Le mail inviate contengono il pdf come allegato.

Mentre se è una bozza viene solo generato il pdf (ma non viene salvato nella cartella) e viene avviato il download sul dispositivo dell’agente.

Se invece si vuole scaricare una copia che è già stata chiusa, non ne viene generata una nuova ma, se presente, viene letta dalla cartella contenente i pdf e inviata per il download. Questo per evitare che eventuali modifiche recenti ai dati possano modificare copie già chiuse in passato.

Il mailsender utilizzato era creato dall’azienda, il tool per la generazione dei pdf invece era **iTextPdf**, entrambi erano salvati sulla repository aziendale per evitare che un cambio di versione possa impedire il corretto funzionamento del programma. Infatti, la versione di iTextPdf indicata nel file pom del progetto era la 5.0.2, sul repository di Maven Central al momento è disponibile solo dalla 5.0.6 in poi. Senza questo accorgimento sarebbero sorti problemi di versionamento.

Le query utilizzate sono praticamente tutte scritte come “native query” con molteplici join poiché vista la complessità e la mole dei dati da ottenere da diverse tabelle, affidarsi solamente alle query semplici di JPA si è rivelato praticamente impossibile.

In certe casistiche i calcoli dei prodotti in omaggio possono variare e quindi anche le query dovevano rispecchiare queste logiche. Inserimmo direttamente nelle query delle condizioni per gestire le variazioni nei calcoli. Questo permise al backend Spring Boot di evitare l’esecuzione di certi calcoli che il database poteva già effettuare con molta più efficienza e rapidità.

Facendo ciò siamo riusciti a ridurre i tempi di risposta dell’API da una media di 12 secondi a soli 2.

Un bonus ulteriore di questa scelta è stata la diminuzione del volume effettivo di dati mandati come risposta alla query.

Il progetto è ancora in corso e probabilmente nel periodo iniziale di lavoro dopo l’assunzione sarà un mio compito continuare a portare avanti questo progetto.

Competenze utilizzate

Hard skills

Parte delle hard skills che ho messo a frutto erano cose che sapevo già e quindi ho principalmente effettuato un consolidamento delle conoscenze su:

Android (java e xml)

- **Location Providers:** I provider forniti dal Sistema android si dividono in tre categorie: Network, GPS e Fused. Ogni tipo di provider ha dei pro e contro e in base alle esigenze uno può essere più o meno adatto degli altri.

I **Network provider** si affiancano ai ripetitori di segnale per triangolare la posizione dell’utilizzatore.

Sono consigliati per l’utilizzo in ambienti urbani e in edifici al chiuso, inoltre hanno un consumo energetico inferiore rispetto al GPS e generalmente hanno un tempo di risposta più rapido.

I difetti principali sono una precisione inferiore che varia tra i 50 e 500m e la dipendenza dalla rete

mobile o wi-fi a cui il dispositivo deve essere connesso.

I **GPS provider**, come suggerisce già il nome, fanno uso del segnale GPS e perciò sono più adatti all'utilizzo in spazi aperti mantenendo una precisione elevata. I difetti principali sono la lentezza nell'acquisizione della prima posizione, consumi energetici più elevati e la difficoltà nell'operare in ambienti chiusi (gallerie) o con numerosi ostacoli (palazzi).

I **Fused provider** offrono un buon compromesso tra GPS e Network bilanciando la loro precisione e il consumo energetico. È in grado di adattarsi in base al contesto e cambiare la fonte delle posizioni in modo automatico. Uno dei suoi svantaggi è la dipendenza dalla presenza dei **Google Play Services** (certi dispositivi potrebbero esserne sprovvisti) ed è meno facile controllarne i comportamenti specifici a basso livello.

- **Gestione dello storage locale:** Le note dell'applicazione per il Rally, una volta inserite vengono salvate in una lista all'interno dei file privati dell'applicazione attraverso una classe `StorageManager`. Questa classe gestisce sia la scrittura che la lettura dei dati. Poiché non sono dati sensibili non abbiamo ritenuto necessario criptarli, optando invece di salvarli in chiaro in formato JSON. I flag per la visibilità degli elementi dell'interfaccia invece vengono salvati nelle **SharedPreferences**.

Spring Boot

- **JWT (JSON Web Token):** Un token di sicurezza che consente l'autenticazione stateless tra client e server. È composto da tre parti: l'**header**, che contiene le informazioni relative al tipo di algoritmo utilizzato per la codifica del token, il **payload**, che contiene le informazioni (chiamate "claims") in formato JSON codificato e una **firma** che certifica l'integrità e l'autenticità del token, proteggendolo dalle manomissioni.
- **API REST:** Un'architettura per servizi web che utilizza i metodi http (generalmente GET, POST, PUT e DELETE) per eseguire le operazioni **CRUD** (CREATE, READ, UPDATE, DELETE).

- **MVC pattern (Model-View-Controller):** Consiste in un pattern architetturale che promuove la separazione delle varie parti del progetto sulla base della loro funzione. Le parti si distinguono in business-logic (**Model**), presentazione dei dati (**View**), controllo del flusso (**Controller**).
Rende il codice più organizzato, testabile e mantenibile nel tempo.
- **Testing:** implementazione di test automatizzati per verificare il corretto funzionamento del codice.
I **test unitari** sono dei tipi di test che si occupano di verificare che i singoli frammenti atomici di codice restituiscano i dati corretti, i **test di integrazione** invece verificano la corretta interazione tra i vari frammenti. Il tutto serve a verificare che modifiche o implementazioni di nuove funzionalità non abbiano influito sul funzionamento di altre parti del progetto.
Quando un progetto ha una **coverage** elevata vuol dire che gran parte del codice ha dei test che ne verificano il funzionamento. Più casistiche vengono testate e gestite, più sarà facile rilevare e risolvere eventuali problemi futuri. Ogni volta che si effettua una nuova implementazione è buona norma creare dei test per la parte aggiunta.
- **Mocking:** Legato al concetto di testing, consiste nel creare versioni false di componenti e ambienti per utilizzarli nei test senza preoccuparsi di dover dipendere da dipendenze esterne o componenti instabili. Questo può essere svolto tramite **Mockito**, uno dei framework più utilizzati per creare mock in Java.

Angular (typescript e Javascript)

- **Router:** è un modulo di Angular che utilizza il concetto di Single Page Application. Permette la navigazione tra diversi componenti dell'applicazione senza dover ricaricare l'intera pagina, consentendo di definire percorsi (chiamati "route") legati a componenti specifici all'interno di un file **app-routing.module.ts**.

- **Toastr:** è una libreria Javascript per creare notifiche non bloccanti. Permette di creare messaggi di notifica che vengono mostrati ai bordi dello schermo senza bloccare le azioni dell'utente come invece succedrebbe usando un semplice Alert. Ne abbiamo fatto uso nel progetto CCE2025 per mostrare i messaggi di errore provenienti dal backend.
- **TailwindCSS:** è un framework che da tante classi già pronte per apportare stili css in modo rapido e intuitivo. Per utilizzarlo basta scrivere nel tag che si vuole modificare le classi dello stile fornite da Tailwind seguite dal valore che si vuole usare. La sintassi è abbastanza semplice e può essere personalizzata dichiarando classi e valori personalizzati nel file tailwind.css.

Esempi di classi Tailwind sono i seguenti:

"bg-black" indica che il colore del background, di norma, sarà nero;

"dark:bg-white" indica che se in modalità dark verrà invece usato il colore bianco;

"hover:text-[#ff0000]" indica che quando si passa il cursore sul componente il testo diventerà rosso;

Inoltre le classi possono essere concatenate come stringhe per dare più proprietà:

"bg-black dark:bg-white text-gray-800 hover:text-[#ff0000]".

- **PrimeIcons:** è una libreria che permette di utilizzare più di 250 icone open source. Lo abbiamo impiegato in vari progetti per via della qualità e varietà delle icone che mette a disposizione e alla sua facilità d'uso. Inoltre, utilizzando il plugin **tailwindcss-primeui** è possibile integrare PrimeIcons con Tailwind per gestire i colori e le caratteristiche delle icone utilizzando le classi di Tailwind.

Database

- **MySQL:** è un database relazionale open source che fa uso del linguaggio SQL (**Structured Query Language**), da cui prende il nome. È noto per la sua stabilità, efficienza e scalabilità, tanto è vero che anche Facebook, Twitter e YouTube ne fanno uso.

MySQL supporta tipi di dati numerici, caratteri, data e ora, spaziali e JSON e vengono immagazzinati in tabelle formate da righe e colonne.

- **PostgreSQL**: è un database relazionale a oggetti, è open source e offre funzionalità aggiuntive rispetto a MySQL. PostgreSQL offre maggiore flessibilità sui tipi di dati che può immagazzinare nelle tabelle. Supporta tutti i tipi di dati MySQL, ma anche indirizzi di rete geometrici ed enumerati, array, intervalli, XML, hstore e compositi. Inoltre, tali oggetti supportano i paradigmi di ereditarietà e mantengono le loro relazioni padre-figlio.

- **H2**: è un database relazionale scritto in Java, noto per essere leggero e veloce ad avviarsi.

Spesso viene utilizzato per effettuare test come sostituto del database reale perché può essere creato **in-memory**. I database in-memory sono volatili, i loro dati sono temporanei, allo spegnimento o riavvio del database ritorna allo stato iniziale.

Per creare un database H2 in-memory per i test ci sono vari modi. Uno di questi consiste nell'inserire nel file pom del nostro progetto la sua dependency, specificando che dovrà essere utilizzato nell'ambito di test, e non a runtime:

```
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>test</scope>
</dependency>
```

E successivamente creare un file `.yaml` con le istruzioni da seguire per avviare il database di test:

```
1  spring:
2    jpa:
3      hibernate:
4        ddl-auto: create-drop
5        show_sql: true
6      properties:
7        hibernate:
8          dialect: org.hibernate.dialect.H2Dialect
9    datasource:
10      url: jdbc:h2:mem:basedb;MODE=MySQL;DB_CLOSE_DELAY=-1
11      driver-class-name: org.h2.Driver
12    h2:
13      console:
14        enabled: true
15    sql:
16      init:
17        platform: h2
18        schema-locations: classpath:/dbH2/create-tables.sql
```

Questo file poi dovrà essere avviato ogni volta che sta per iniziare un nuovo test in modo tale da essere sicuri che i test successivi utilizzino dati non modificati dai test precedenti.

Novità

- **GitHub vs GitLab:** GitHub e GitLab sono piattaforme di gestione del codice sorgente basate su **Git**, il sistema di versionamento distribuito creato da Linus Torvalds nel 2005 per lo sviluppo del kernel Linux. Git lavora con i cosiddetti **repository**, in cui gli utenti caricano il codice sorgente che possono poi elaborare insieme nel browser, negli editor di codice o in un terminale.

Entrambe consentono a team di sviluppatori di lavorare in contemporanea sullo stesso progetto, tenere traccia delle modifiche, gestire rollback e mantenere una visione chiara dell'evoluzione del codice.

Nonostante siano due software diversi condividono alcune funzionalità:

- Hosting di repository Git (pubblici e privati).
- Collaborazione tramite merge/pull request.
- Supporto a CI/CD pipelines.
- Sistema di issue tracking.
- Interfaccia web moderna e integrazione con editor/IDE.
- Notifiche, review, commenti e gestione dei conflitti.



Per molto tempo, un grande vantaggio di GitLab è stato costituito dal numero indefinito di repository gratuiti a disposizione degli utenti. Su questo fronte GitHub ha recuperato terreno e ora offre anche questa funzionalità. Ciononostante, GitHub impone nella versione gratuita più restrizioni rispetto a GitLab. I rami protetti, ossia i rami di sviluppo ai quali hanno accesso solo determinati utenti, possono ad esempio essere utilizzati sia in GitLab che in GitHub. In GitHub, tuttavia, solo in repository pubblici, mentre in GitLab questa funzione è disponibile anche con repository privati. Ancora più pesante è la limitazione a massimo tre sviluppatori per repository privato in GitHub. Chi desidera lavorare in team più grandi deve passare alla versione Enterprise. In generale, GitHub offre un po' meno permessi utente: in GitHub, una gestione dei permessi strutturata in base a diversi ruoli è possibile solo con l'abbonamento team a pagamento, mentre in

GitLab quest'opzione è standard. Inoltre, GitLab offre un Container-Registry con cui memorizzare e gestire le immagini **Docker** direttamente all'interno del repository, semplificando la gestione dei container nei progetti che utilizzano architetture a microservizi.

Entrambe le piattaforme sono valide e ampiamente usate nel settore dello sviluppo software.

Tuttavia, GitLab risulta spesso preferibile in ambito aziendale, soprattutto per la possibilità di hosting locale gratuito, l'integrazione nativa di strumenti DevOps, e la maggiore flessibilità nella gestione di permessi e collaboratori.

GitHub, invece, è molto popolare nel mondo open-source e ha una community vastissima, rendendolo ideale per progetti pubblici e contributi esterni.

- **Docker:** è una piattaforma open source che permette di eseguire applicazioni (**immagini**) all'interno di **container**. L'immagine è un template immutabile di un'applicazione che contiene le informazioni



necessary a Docker per creare i container. Un container è un'istanza creata dall'immagine, una sorta di "scatola" che contiene tutto il necessario per far funzionare il programma che ha al suo interno.

Come funzionamento è simile a una **macchina virtuale** ma ha il vantaggio di non avere bisogno di virtualizzare tutto un sistema operativo, fa uso solo del kernel della macchina che lo ospita e questo lo rende più rapido ad avviarsi, portatile e leggero.

Docker è particolarmente indicato per far funzionare applicazioni a microservizi poiché ogni microservizio può essere containerizzato e moltiplicato all'occorrenza.

Quando serve **scalabilità orizzontale**, ad esempio nei casi delle applicazioni con molti utenti attivi allo stesso tempo, anziché avere una sola istanza che gestisce tutto il carico proveniente dagli utenti possiamo invece avviare più istanze e grazie a un **load-balancer** riuscire a distribuire il traffico in modo più efficiente.

Inoltre, se uno dei container fallisce gli altri rimangono attivi e quindi consente di mantenere gran parte del sistema ancora in funzione.

Per creare un container da una image generalmente serve un file docker-compose.

```
1  #file to tell docker how to create a container
2  #run with "docker-compose up -d" in cmd from the same folder path as this file
3  ►► services:
4  ►  pg-service:
5      restart: unless-stopped
6      image: postgres:latest
7      environment:
8          - POSTGRES_USER=postgres
9          - POSTGRES_PASSWORD=postgres
10     volumes:
11         - ./create-database.sql:/docker-entrypoint-initdb.d/create-database.sql
12         - pg-volume:/var/lib/postgresql/data
13     ports:
14         - "5432:5432"
15
16     volumes:
17         pg-volume:
```

Nel file compose è indicato in **services** i container che si vogliono creare, **volumes** dichiara quale volume Docker si vuole usare per salvare i dati persistenti dei container, **restart:unless-stopped** vuol dire che il container proverà a riavviarsi automaticamente in caso di fallimento, **image:postgres:latest** indica l'immagine da utilizzare (in questo caso la versione più recente di PostgreSQL presa da Docker Hub), **environment** contiene le variabili ambientali (in questo caso contiene le credenziali di default per l'accesso al database), **volumes** (quello all'interno di services) esegue dei file SQL che crea le tabelle all'avvio del database e le salva nel volume pg-volume, in fine **ports:"5432:5432"** associa la porta di Postgres a quella della macchina.

Soft skills

- **Rubber-duck debugging**: è una metodologia utile all'ispezione del codice ma è applicabile anche a molti ambiti di vita quotidiana. Consiste nello spiegare in modo semplificato, come se si stesse parlando con qualcuno che dell'argomento non sa niente, tutti gli step che il programma dovrebbe eseguire durante la sua esecuzione.

La spiegazione generalmente non viene fatta a dei colleghi in carne ed ossa, per evitare di interromperli nel loro lavoro, ma a un oggetto inanimato (ad esempio una **peperella di gomma**).

Quando si spiega un argomento spesso lo si inizia a vedere da prospettive diverse e quindi aiuta a

comprendere più nel dettaglio cosa sta succedendo, cosa non sta funzionando, perché non sta funzionando e come aggiustarlo.

Molte volte mi è capitato di incontrare errori generici che mi sembravano senza senso, per poi scoprire che erano solo degli errori sciocchi o piccole sviste.

- **Pair programming:** è una tecnica di sviluppo di codice con la quale due programmatori lavorano sulla stessa postazione. Nella coppia uno dei due è incaricato di scrivere il codice (e prende il nome di **driver**), l'altro invece controlla ogni riga di codice nel mentre viene scritta e fornisce feedback e miglioramenti da apportare (assume il ruolo chiamato **navigator** o **observer**), i due poi si scambiano di ruolo a turni.

Può sembrare un po' strano ma dividendosi i compiti chi scrive riesce a concentrarsi meglio e essere più produttivo, invece chi fa da guida lo supporta e quindi riduce di molto la probabilità che si presentino errori in futuro.

Questa tecnica inizialmente raddoppia la quantità di persone necessarie per la scrittura del codice (e quindi anche i costi) ma è anche da considerare come un investimento perché poi sarà necessario spendere molto meno tempo per le correzioni dovute ai controlli di qualità (quindi teoricamente si riscontrerà un risparmio).

Sfide affrontate e lezioni imparate

- **H2 e la sua sintassi pignola:** Durante lo sviluppo di un progetto, ho utilizzato H2 come database in memoria per eseguire i test automatici, in modo da non toccare i dati reali presenti sul MySQL. Nonostante nel file pom.xml avessi configurato correttamente la dipendenza e specificato che H2 dovesse essere usato solo nella fase di test, mi sono accorto che i test continuavano a girare sul database MySQL vero, causando problemi come la modifica o la cancellazione accidentale di dati. Dopo varie prove e ricerche, ho scoperto che la causa era un'incompatibilità nella sintassi SQL.

Infatti stavo provando a utilizzare con H2 lo stesso script di creazione del database reale di MySQL.

Il problema era che H2, rispetto a MySQL, è molto più rigido nel parsing degli script SQL e in certi casi necessita anche una sintassi diversa.

Quando giungeva il momento di effettuare i test, H2 non riusciva a creare un nuovo database e a popolarlo correttamente con gli script che gli avevo fornito perché non erano scritti esattamente nel modo che voleva lui. Questo costringeva Spring Boot a ricorrere all'utilizzo di MySQL per effettuare i test, poiché era l'unico database a sua disposizione in quel momento.

Inoltre la mancanza di un “;” alla fine di un paio di istruzioni SQL generava ulteriori problemi poiché nonostante MySQL in questi casi sia abbastanza tollerante, per H2 la presenza dei “;” è tassativa.

- **L'importanza delle best-practices e degli standard:** Durante lo sviluppo del progetto CCE2025, ho potuto toccare con mano quanto possa essere dannoso iniziare un progetto senza una visione d'insieme condivisa.

All'inizio, io e gli altri stagisti abbiamo riscritto da zero diverse parti del codice, ma senza definire uno standard comune. Questo ci ha permesso, nel breve termine, di procedere velocemente: ognuno lavorava su una funzionalità e la integrava nel progetto tramite una branch personale derivata da develop. Tuttavia, questa mancanza di coerenza strutturale si è fatta sentire col passare del tempo: con l'aumentare del numero di feature e merge, la situazione è rapidamente degenerata.

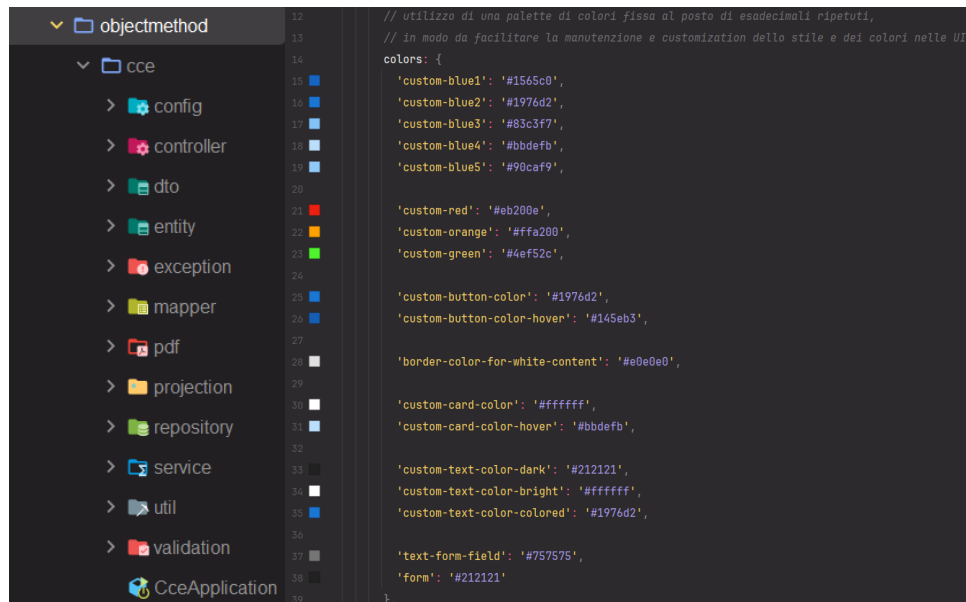
Ci siamo trovati a dover affrontare:

- Merge conflittuali complessi
- Confusione nell'organizzazione delle cartelle, con file disposti secondo criteri diversi
- Entità incomplete o assenti, dato che in molte situazioni utilizzavamo solo i DTO per semplificare, vista l'assenza di vincoli come le foreign key

Questa situazione ha complicato in particolare la scrittura di query complesse con join tra tabelle, poiché la struttura dati non era solida né, a volte, coerente.

A un certo punto decidemmo di fermarci e di definire delle linee guida comuni, le quali hanno migliorato nettamente la qualità e la manutenibilità del codice.

Per il **backend**, scegliemmo una struttura a pacchetti raggruppata per tipo (es: dto/ClienteDTO.java, service/ ClienteService.java, ecc.) anziché una struttura per dominio (es: cliente/ClienteDTO.java, cliente/ClienteService.java, ecc.).



Per il **frontend**, creammo una palette di colori unificata con TailwindCSS e li sostituimmo dove necessario, in modo che una modifica alla palette centrale si potesse propagare automaticamente in tutto il progetto.

Partire subito fa risparmiare tempo all’inizio, ma senza standard condivisi, la complessità cresce in modo incontrollato e rallenta il flusso nel medio-lungo termine. Fermarsi per allinearsi sugli standard è un investimento che paga nel medio-lungo termine e questo lo abbiamo capito a nostre spese.

- **Avere a che fare con progetti “Legacy”:** Nel progetto CCE2025, ci siamo trovati a lavorare su un'applicazione originariamente sviluppata oltre vent’anni fa. Il nostro compito consisteva nel

riscrivere l'intera applicazione utilizzando tecnologie moderne, mantenendo però invariata la logica di funzionamento e senza modificare il database, poiché era ancora utilizzato da altri sistemi aziendali. Questa situazione ha presentato diverse sfide tipiche dei progetti legacy:

Assenza di foreign key dichiarate: il database conteneva colonne che "sembravano" chiavi esterne per convenzione di nome, ma nessun vincolo era effettivamente dichiarato. Questo ci ha inizialmente portati a usare solo i DTO senza creare le relative entità, per semplificare lo sviluppo.

Nomi delle colonne poco chiari: in diversi casi, il nome di un campo non rappresentava chiaramente il suo contenuto o il suo scopo, creando ambiguità.

Logiche di calcolo complesse e non documentate: alcune operazioni, in particolare quelle legate alle commissioni e copie commissioni, restituivano valori calcolati in modi poco comprensibili e non sempre prevedibili. Questo ci ha costretto spesso a confrontarci direttamente con i referenti funzionali per comprendere il significato dei dati.

Lavorare su un sistema legacy richiede molta pazienza, spirito di adattamento e una buona capacità di analisi. A volte il codice da scrivere è il meno dei problemi: la vera sfida è capire il sistema esistente, con i suoi limiti e le sue particolarità.

Conclusioni e prospettive future

L'ambiente in ufficio mi è sembrato molto umano, serio quando serve ma anche allegro nei momenti di relax, come ad esempio le partite al calcio balilla con il titolare dell'azienda durante le pause pranzo. Inoltre, i colleghi erano molto disponibili, competenti e persino con hobby simili ai miei.

A inizio stage non avevo ancora idea di cosa fare dopo il biennio in ITS, ma ora che lo stage in Object Method è ormai giunto al termine la scelta di lanciarmi su questo tipo di carriera mi sta iniziando a convincere sempre più.

L'azienda mi ha offerto un contratto di apprendistato da programmatore quindi penso proprio che continuerò per questa strada.

Il mondo della programmazione fino ad adesso mi ha mostrato un mondo creativo e ho intenzione di ampliare le mie conoscenze negli anni a venire. Primo tra tutti probabilmente vorrei iniziare a imparare C++ per via della sua elevata versatilità in molti ambiti, nonché la sua efficienza e poi magari avvicinarmi anche verso la creazione di videogames.

Nessuno sa cosa ci tiene in serbo il futuro ma è mia intenzione scoprirlo giorno per giorno.

Sitografia

<http://www.objectmethod.cloud/>

<https://www.itsrizzoli.it/>

<https://www.iisbassi.edu.it/pagine/its-rizzoli>

<https://www.iisbassi.edu.it/>

https://en.wikipedia.org/wiki/Rubber_duck_debugging

https://en.wikipedia.org/wiki/Pair_programming

<https://docs.spring.io/spring-boot/reference/io/quartz.html>

<https://developers.google.com/location-context/fused-location-provider?hl=it>

<https://github.com/mockito/mockito>

<https://github.com/CodeSeven/toastr>

<https://v17.angular.io/api/router>

<https://primeng.org/icons>

<https://aws.amazon.com/it/compare/the-difference-between-mysql-vs-postgresql/>

<https://www.baeldung.com/spring-boot-h2-database>

<https://docs.docker.com/>

<https://it.wikipedia.org/wiki/Bcrypt>